

RetroSpectrum.c DSP Mathematics and Implementation Reference

August 11, 2026

Abstract

This document explains the digital signal processing and RF calculations implemented in the supplied `RetroSpectrum.c`. For every major calculation, it shows the relevant C code, expresses the same operation mathematically, defines the variables, and explains why the calculation is performed. The emphasis is the live receive/spectrum path and the selected-signal recording path: normalized IQ, DC rejection, FFT windowing, FFT shifting, magnitude-to-decibel conversion, frequency translation, low-pass FIR design, decimation, and signed 16-bit IQ quantization. A final section covers the quarter-wave antenna calculation and transmit-side sample-format conversion.

Contents

1	Scope and Signal Model	3
1.1	Important default values	3
2	Selector Geometry to RF Frequency	3
2.1	Selected center frequency: <code>selection_center_Hz()</code>	3
2.2	Selected bandwidth: <code>selection_BW_Hz()</code>	4
3	Receive Sample Normalization and DC Rejection	4
3.1	CS16 to normalized floating point: <code>process_rx_samples()</code>	4
3.2	DC estimate and subtraction	5
4	Live Spectrum FFT Pipeline	6
4.1	Hann window construction in <code>main()</code>	6
4.2	Applying the window and 50% overlap: <code>ring_read_block()</code>	6
4.3	Forward FFT	6
4.4	FFT shift and dB conversion: <code>compute_DB_from_FFT()</code>	7
4.4.1	FFT shift	7
4.4.2	Complex magnitude	7
4.4.3	Magnitude to decibels	7
4.4.4	Hann coherent-gain note	7
5	Selected-Signal Recording Filter Design	8
5.1	Overview of <code>configure_recording_filter()</code>	8
5.2	Requested output sample rate	8
5.3	Integer decimation factor	8
5.4	Actual output sample rate	8
5.5	Low-pass cutoff	9

5.6	Normalized cutoff	9
5.7	Ideal sinc low-pass impulse response	9
5.8	Hamming window for the FIR	10
5.9	DC-gain normalization	10
6	Frequency Translation, FIR Filtering, and Decimation	10
6.1	Selected-center translation: <code>recorder_write_sample()</code>	10
6.2	Phase wrapping	11
6.3	No-decimation path	12
6.4	Circular FIR history	12
6.5	Decimation gate	12
6.6	FIR convolution at output instants	12
7	Complex16 Quantization and Queue Conversions	13
7.1	Float to signed 16-bit IQ	13
7.2	Queue/cache reconstruction back to float	13
7.3	Five-second pre-record cache capacity	14
8	End-to-End Recording Equation	14
9	Worked Recording Example	14
9.1	Selected center	15
9.2	Selected bandwidth	15
9.3	Target rate and decimation	15
9.4	Filter cutoff	15
9.5	Mixer phase step	15
10	Quarter-Wave Antenna Calculation	15
11	Transmit-Side CS16 to CF32 Conversion	16
12	Engineering Notes and Interpretation	16
13	Compact Formula Reference	17
A	Full Core DSP Function Listings	17
A.1	<code>selection_center_Hz()</code>	17
A.2	<code>selection_BW_Hz()</code>	18
A.3	<code>configure_recording_filter()</code>	18
A.4	<code>ring_read_block()</code>	19
A.5	<code>compute_DB_from_FFT()</code>	19
A.6	Core calculation portion of <code>process_rx_samples()</code>	19
A.7	<code>recorder_write_sample()</code>	20
A.8	<code>recommended_antenna_length_inches()</code>	21

1 Scope and Signal Model

The core complex baseband sample is represented as

$$x[n] = I[n] + jQ[n], \quad (1)$$

where $I[n]$ is the in-phase component and $Q[n]$ is the quadrature component. In RetroSpectrum, most DSP calculations operate on floating-point samples nominally bounded to $[-1, 1]$.

The major receive path is

SDR IQ $\rightarrow$ normalization $\rightarrow$ optional DC rejection
 $\rightarrow$ ring buffer $\rightarrow$ Hann window $\rightarrow$ FFT
 $\rightarrow$ FFT shift $\rightarrow$ magnitude/dB.

The selected-signal recording path is

SDR IQ $\rightarrow$ selected-center translation to DC
 $\rightarrow$ low-pass FIR $\rightarrow$ decimation
 $\rightarrow$ complex16 quantization.

1.1 Important default values

The source defines the following relevant defaults and constants:

```
124 #define DEFAULT_CENTER_FREQ_HZ 101300000ULL
125 #define DEFAULT_SAMPLE_RATE_HZ 2000000U
126 #define DEFAULT_DISPLAY_SPAN_HZ 1000000U
145 #define PRE_RECORD_SECONDS 5
149 #define REC_FIR_TAPS 255
```

Thus, by default,

$$f_c = 101.3 \text{ MHz}, \quad (2)$$

$$F_s = 2.0 \text{ MHz}, \quad (3)$$

$$B_{\text{display}} = 1.0 \text{ MHz}, \quad (4)$$

$$M_{\text{FIR}} = 255 \text{ taps}. \quad (5)$$

2 Selector Geometry to RF Frequency

The selector stores horizontal positions as normalized fractions of the displayed spectrum. A position of 0 is the left edge, 0.5 is the center, and 1 is the right edge.

2.1 Selected center frequency: selection_center_Hz()

Relevant code (RetroSpectrum.c lines 471–489):

```
477 double Center_Frac = (Global_Selector.X0 + Global_Selector.X1) * 0.5;
479 double Offset_Hz = (Center_Frac - 0.5) * (double)Global_Display_Span_Hz;
481 double Calc_Freq = (double)Global_Center_Freq_Hz + Offset_Hz;
```

The selector center is the midpoint of its two horizontal boundaries:

$$x_{\text{sel}} = \frac{X_0 + X_1}{2}. \quad (6)$$

The normalized display coordinate is then recentered around zero:

$$x_{\text{offset}} = x_{\text{sel}} - \frac{1}{2}. \quad (7)$$

This converts the display range

$$[0, 1] \rightarrow \left[-\frac{1}{2}, +\frac{1}{2}\right]. \quad (8)$$

Multiplying by the total displayed span converts the normalized offset to hertz:

$$\Delta f = \left(x_{\text{sel}} - \frac{1}{2}\right) B_{\text{display}}. \quad (9)$$

Finally,

$$f_{\text{sel}} = f_c + \Delta f. \quad (10)$$

Therefore, $x_{\text{sel}} = 0.5$ produces $\Delta f = 0$, while $x_{\text{sel}} = 0.75$ produces an offset of $+0.25B_{\text{display}}$.

2.2 Selected bandwidth: selection_BW_Hz()

Relevant code (RetroSpectrum.c lines 492–515):

```

498 double BW = fabs(Global_Selector.X1 - Global_Selector.X0) * (double)Global_Display_Span_Hz;
500 if (BW < 1000.0) {
502     BW = 1000.0;
504 }
506 if (BW > (double)Global_Sample_Rate_Hz) {
508     BW = (double)Global_Sample_Rate_Hz;
510 }

```

The normalized selector width is

$$w = |X_1 - X_0|. \quad (11)$$

The RF bandwidth represented by that width is

$$B_{\text{sel}} = w B_{\text{display}}. \quad (12)$$

RetroSpectrum then enforces

$$1000 \text{ Hz} \leq B_{\text{sel}} \leq F_s. \quad (13)$$

The lower clamp avoids a zero or impractically tiny recording bandwidth, while the upper clamp prevents the selected bandwidth from exceeding the sampled bandwidth.

3 Receive Sample Normalization and DC Rejection

3.1 CS16 to normalized floating point: process_rx_samples()

Relevant code (RetroSpectrum.c lines 2208–2306):

```

2235 if (samples_cs16) {
2237     I = (float)samples_cs16[2U * sample] / 32768.0f;
2238     Q = (float)samples_cs16[2U * sample + 1U] / 32768.0f;
2240 }
2242 else {
2244     I = samples_cf32[2U * sample];
2245     Q = samples_cf32[2U * sample + 1U];
2247 }

```

For signed 16-bit input,

$$I_f = \frac{I_{16}}{32768}, \quad Q_f = \frac{Q_{16}}{32768}. \quad (14)$$

Since a signed 16-bit integer spans $[-32768, 32767]$, this maps to approximately

$$[-1, 0.9999695]. \quad (15)$$

CF32 input is already floating-point IQ and is copied directly.

The function also replaces non-finite values with zero:

```
2249 if (!isfinite(I)) I = 0.0f;
2255 if (!isfinite(Q)) Q = 0.0f;
```

This prevents NaN or infinity from propagating into the FFT, filters, waterfall, and recording path.

3.2 DC estimate and subtraction

When DC correction is enabled, the source performs:

```
2263 const double alpha = 0.0001;
2265 Global_DC_I += alpha * ((double)I - Global_DC_I);
2266 Global_DC_Q += alpha * ((double)Q - Global_DC_Q);
2268 I -= (float)Global_DC_I;
2269 Q -= (float)Global_DC_Q;
```

For either component, let $x[n]$ be the incoming sample and $d[n]$ the running DC estimate. The update is

$$d[n] = d[n-1] + \alpha(x[n] - d[n-1]), \quad (16)$$

which can be rewritten as

$$d[n] = (1 - \alpha)d[n-1] + \alpha x[n]. \quad (17)$$

This is a one-pole exponential moving-average low-pass filter. The output used by RetroSpectrum is

$$y[n] = x[n] - d[n]. \quad (18)$$

Thus, a slowly varying or constant component is estimated by $d[n]$ and removed from $x[n]$.

The exact z -domain transfer function of the resulting DC blocker is

$$H(z) = \frac{Y(z)}{X(z)} = \frac{(1 - \alpha)(1 - z^{-1})}{1 - (1 - \alpha)z^{-1}}. \quad (19)$$

At DC, $z = 1$, so

$$H(1) = 0, \quad (20)$$

which is the desired null at zero frequency.

For small α , the low-frequency corner is approximately

$$f_{\text{corner}} \approx \frac{-\ln(1 - \alpha)}{2\pi} F_s \approx \frac{\alpha F_s}{2\pi}. \quad (21)$$

With $\alpha = 10^{-4}$ and $F_s = 2 \text{ MHz}$,

$$f_{\text{corner}} \approx 31.8 \text{ Hz}. \quad (22)$$

This is slow enough to remove a DC offset while leaving ordinary RF modulation largely unaffected.

4 Live Spectrum FFT Pipeline

4.1 Hann window construction in main()

Relevant code (RetroSpectrum.c lines 5847–5851):

```
5847 for (int n = 0; n < FFT_SIZE; n++) {
5848     hann_window[n] = 0.5 - 0.5 * cos((2.0 * M_PI * n) / (FFT_SIZE - 1));
5849 }
5851 fftw_plan plan = fftw_plan_dft_1d(FFT_SIZE, time_domain, freq_domain, FFTW_FORWARD, FFTW_MEASURE);
```

For an FFT length $N = \text{FFT_SIZE}$, the Hann window is

$$w[n] = \frac{1}{2} - \frac{1}{2} \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n < N. \quad (23)$$

The window tapers the edges of each finite FFT block toward zero. This reduces spectral leakage caused by treating a finite block as though it repeats periodically.

4.2 Applying the window and 50% overlap: ring_read_block()

Relevant code (RetroSpectrum.c lines 1683–1714):

```
1698 for (int sam = 0; sam < FFT_SIZE; sam++) {
1699     size_t idx = (r->read_pos + sam) % RING_SIZE;
1700     in[sam][0] = r->I[idx] * window[sam];
1701     in[sam][1] = r->Q[idx] * window[sam];
1702 }
1708 r->read_pos = (r->read_pos + FFT_SIZE / 2) % RING_SIZE;
```

The complex windowed sequence is

$$x_w[n] = x[n]w[n]. \quad (24)$$

Equivalently,

$$I_w[n] = I[n]w[n], \quad (25)$$

$$Q_w[n] = Q[n]w[n]. \quad (26)$$

After each FFT block, the read pointer advances by $N/2$ samples rather than N samples. Therefore the overlap is

$$1 - \frac{N/2}{N} = \frac{1}{2} = 50\%. \quad (27)$$

The next FFT reuses half of the previous block. This increases time continuity in the waterfall and is a standard pairing with the Hann window.

4.3 Forward FFT

FFTW computes the forward discrete Fourier transform. Mathematically,

$$X[k] = \sum_{n=0}^{N-1} x_w[n] e^{-j2\pi kn/N}, \quad 0 \leq k < N. \quad (28)$$

The FFT converts the time-domain complex IQ block into complex frequency bins.

The nominal raw FFT-bin spacing is

$$\Delta f_{\text{bin}} = \frac{F_s}{N}. \quad (29)$$

Before an FFT shift, bins $0, 1, \dots$ begin at DC and progress through positive frequencies, while negative frequencies are stored in the upper half of the FFT array.

4.4 FFT shift and dB conversion: compute_DB_from_FFT()

Relevant code (RetroSpectrum.c lines 2374–2389):

```
2381 int shifted_sam = (sam + FFT_SIZE / 2) % FFT_SIZE;
2383 double I = out[shifted_sam][0];
2384 double Q = out[shifted_sam][1];
2386 double magnitude = sqrt(I * I + Q * Q) / FFT_SIZE;
2387 db[sam] = 20.0 * log10(magnitude + 1e-12) + 100.0;
```

4.4.1 FFT shift

The index transformation is

$$k' = \left(k + \frac{N}{2}\right) \bmod N. \quad (30)$$

This rearranges the bins so the negative-frequency half appears first, DC is in the middle, and the positive-frequency half appears last:

$$[-F_s/2, \dots, 0, \dots, +F_s/2). \quad (31)$$

This layout matches the conventional SDR spectrum display centered on the tuned RF center frequency.

4.4.2 Complex magnitude

For a complex FFT bin

$$X[k] = I_k + jQ_k, \quad (32)$$

its magnitude is

$$|X[k]| = \sqrt{I_k^2 + Q_k^2}. \quad (33)$$

RetroSpectrum divides by the FFT length:

$$A[k] = \frac{|X[k]|}{N}. \quad (34)$$

4.4.3 Magnitude to decibels

The display calculation is

$$D[k] = 20 \log_{10} (A[k] + 10^{-12}) + 100. \quad (35)$$

The 10^{-12} term prevents $\log_{10}(0)$ from producing negative infinity. The +100 term is a display offset that raises the plotted values into a convenient range.

Important: this value is a relative display quantity, not calibrated dBm. No impedance calibration, SDR gain calibration, or absolute power-reference conversion appears in this function.

4.4.4 Hann coherent-gain note

The code divides by N but does not compensate for the Hann window's coherent gain. A Hann window has a mean near 0.5, so a bin-centered unit complex sinusoid produces approximately

$$A \approx 0.5, \quad (36)$$

and therefore

$$20 \log_{10}(0.5) \approx -6.02 \text{ dB} \quad (37)$$

before the display offset. This is acceptable for a relative waterfall, but it is another reason the result should not be interpreted as calibrated absolute power.

5 Selected-Signal Recording Filter Design

5.1 Overview of `configure_recording_filter()`

This function designs the anti-alias low-pass FIR and chooses an integer decimation factor. It operates after the selected signal is translated to zero frequency by `recorder_write_sample()`.

Relevant source range: RetroSpectrum.c lines 517–609.

5.2 Requested output sample rate

```
533 double wanted_out_rate = (double)Global_Rec_BW_Hz * 3;
535 if (wanted_out_rate < 48000.0) {
537     wanted_out_rate = 48000.0;
539 }
```

Let B be the selected bandwidth. The target output rate is

$$F_{s,\text{target}} = \max(3B, 48000). \quad (38)$$

The factor of three provides sampling and transition-band headroom beyond the selected bandwidth. The implementation uses 3x the bandwidth;

5.3 Integer decimation factor

```
541 Global_Rec_Decimation = (int)((double)Global_Sample_Rate_Hz / wanted_out_rate);
543 if (Global_Rec_Decimation < 1) {
545     Global_Rec_Decimation = 1;
547 }
```

The integer decimation factor is

$$D = \max\left(1, \left\lfloor \frac{F_s}{F_{s,\text{target}}} \right\rfloor\right). \quad (39)$$

Using the floor rather than rounding keeps the actual output rate from dropping below the requested target solely due to decimation-factor rounding.

5.4 Actual output sample rate

```
549 Global_Rec_Out_Rate_Hz = Global_Sample_Rate_Hz / (uint32_t)Global_Rec_Decimation;
```

The resulting rate is approximately

$$F_{s,\text{out}} \approx \frac{F_s}{D}. \quad (40)$$

Because the C expression uses integer types, the stored value is the integer quotient

$$F_{s,\text{out}} = \left\lfloor \frac{F_s}{D} \right\rfloor. \quad (41)$$

5.5 Low-pass cutoff

```

554 double cutoff_hz = (double)Global_Rec_BW_Hz * 0.5;
558 double max_safe_cutoff = (double)Global_Rec_Out_Rate_Hz * 0.45;
560 if (cutoff_hz > max_safe_cutoff) {
562     cutoff_hz = max_safe_cutoff;
564 }

```

Once the selected RF center has been mixed to DC, the desired complex signal nominally spans

$$-\frac{B}{2} \leq f \leq +\frac{B}{2}. \quad (42)$$

Therefore the natural low-pass cutoff is

$$f_c = \frac{B}{2}. \quad (43)$$

However, after decimation the Nyquist limit is $F_{s,\text{out}}/2$. RetroSpectrum keeps the cutoff below this limit using 45% of the output sample rate:

$$f_{c,\text{safe}} = 0.45F_{s,\text{out}}. \quad (44)$$

The actual FIR cutoff is

$$f_c = \min\left(\frac{B}{2}, 0.45F_{s,\text{out}}\right). \quad (45)$$

The $0.45F_{s,\text{out}}$ term acts as a safety cap that prevents the FIR cutoff from approaching the post-decimation Nyquist frequency. Under the normal 3x bandwidth rule, the $B/2$ cutoff is typically much lower and provides substantially more transition-band margin.

5.6 Normalized cutoff

```

568 double fc = cutoff_hz / (double)Global_Sample_Rate_Hz;

```

The FIR operates on the original input stream before samples are discarded, so the normalized cutoff is relative to the original input sample rate:

$$f_n = \frac{f_c}{F_s}. \quad (46)$$

Here f_n is in cycles per sample. Nyquist corresponds to $f_n = 0.5$.

5.7 Ideal sinc low-pass impulse response

```

571 int mid = REC_FIR_TAPS / 2;
573 for (int n = 0; n < REC_FIR_TAPS; n++) {
574     int m = n - mid;
576     double sinc;
578     if (m == 0) {
580         sinc = 2.0 * fc;
582     }
584     else {
586         sinc = sin(2.0 * M_PI * fc * (double)m) / (M_PI * (double)m);
588     }
590     ...
596 }

```

With $M = 255$ taps and midpoint

$$n_0 = \left\lfloor \frac{M}{2} \right\rfloor = 127, \quad (47)$$

the centered time index is

$$m = n - n_0. \quad (48)$$

The ideal discrete-time low-pass impulse response is

$$h_{\text{ideal}}[m] = \begin{cases} 2f_n, & m = 0, \\ \frac{\sin(2\pi f_n m)}{\pi m}, & m \neq 0. \end{cases} \quad (49)$$

This is the sampled sinc impulse response of an ideal brick-wall low-pass filter. The special $m = 0$ case is the finite limit of the sinc expression and avoids division by zero.

5.8 Hamming window for the FIR

```
592 double window = 0.54 - 0.46 * cos((2.0 * M_PI * (double)n) / (double)(REC_FIR_TAPS - 1));
594 Global_Rec_FIR[n] = sinc * window;
```

The Hamming window is

$$w_H[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M-1}\right), \quad 0 \leq n < M. \quad (50)$$

The practical FIR coefficients are

$$h[n] = h_{\text{ideal}}[n - n_0]w_H[n]. \quad (51)$$

The ideal sinc is infinitely long; multiplying it by the finite Hamming window produces a realizable 255-tap FIR and reduces the sidelobes caused by abrupt truncation.

5.9 DC-gain normalization

```
595 sum += Global_Rec_FIR[n];
...
600 if (fabs(sum) > 1e-12) {
602     for (int n = 0; n < REC_FIR_TAPS; n++) {
603         Global_Rec_FIR[n] /= sum;
604     }
606 }
```

Before normalization, let

$$S = \sum_{n=0}^{M-1} h[n]. \quad (52)$$

The final coefficients are

$$\hat{h}[n] = \frac{h[n]}{S}. \quad (53)$$

Therefore

$$\sum_{n=0}^{M-1} \hat{h}[n] = 1. \quad (54)$$

For a low-pass FIR, the sum of the coefficients is the frequency response at DC, so this normalization sets the DC gain to unity.

6 Frequency Translation, FIR Filtering, and Decimation

6.1 Selected-center translation: recorder_write_sample()

Relevant code (RetroSpectrum.c lines 1221–1373):

```

1235 double Freq_Offset_Hz = (double)Global_Rec_Center_Hz - (double)Global_Center_Freq_Hz;
1236 double Phase_Step = -2.0 * M_PI * Freq_Offset_Hz / (double)Global_Sample_Rate_Hz;
1238 double C = cos(Global_Rec_Phase);
1239 double S = sin(Global_Rec_Phase);
1241 double Shifted_I = I * C - Q * S;
1242 double Shifted_Q = I * S + Q * C;
1244 Global_Rec_Phase += Phase_Step;

```

The selected signal is offset from the SDR center by

$$\Delta f = f_{\text{rec}} - f_c. \quad (55)$$

The required phase change per input sample is

$$\Delta\phi = -2\pi \frac{\Delta f}{F_s}. \quad (56)$$

The negative sign shifts a signal at $+\Delta f$ down by Δf so that it lands at zero frequency.

At sample n , let the accumulated phase be $\phi[n]$. The complex multiplication is

$$x'[n] = x[n]e^{j\phi[n]}. \quad (57)$$

Expanding into real and imaginary parts gives

$$I'[n] = I[n] \cos \phi[n] - Q[n] \sin \phi[n], \quad (58)$$

$$Q'[n] = I[n] \sin \phi[n] + Q[n] \cos \phi[n]. \quad (59)$$

These are exactly the two calculations in the C source.

Ignoring phase wrapping,

$$\phi[n] \approx -2\pi \frac{\Delta f}{F_s} n, \quad (60)$$

so

$$x'[n] = x[n]e^{-j2\pi\Delta f n/F_s}. \quad (61)$$

Multiplication by this complex exponential translates the spectrum by $-\Delta f$.

6.2 Phase wrapping

```

1246 if (Global_Rec_Phase > M_PI) {
1248     Global_Rec_Phase -= 2.0 * M_PI;
1250 }
1252 if (Global_Rec_Phase < -M_PI) {
1254     Global_Rec_Phase += 2.0 * M_PI;
1256 }

```

Because sine and cosine are 2π -periodic,

$$e^{j(\phi+2\pi k)} = e^{j\phi}, \quad k \in \mathbb{Z}. \quad (62)$$

Keeping the phase near $[-\pi, \pi]$ prevents the accumulator from growing without bound while preserving the same oscillator value.

6.3 No-decimation path

If $D \leq 1$, the function does not run the recording FIR:

```

1258 if (Global_Rec_Decimation <= 1) {
    ...
1286 iq_pair[0] = (int16_t)(Shifted_I * 32767.0);
1287 iq_pair[1] = (int16_t)(Shifted_Q * 32767.0);
1289 fwrite(iq_pair, sizeof(int16_t), 2, Global_Rec_File);
1290 return;
1292 }
```

This is safe from a decimation-aliasing standpoint because no samples are being discarded. It also means that when $D = 1$, the selected bandwidth is not enforced by this FIR; the recording contains the frequency-shifted full sampled bandwidth.

6.4 Circular FIR history

```

1296 Global_Rec_Hist_I[Global_Rec_FIR_Pos] = Shifted_I;
1297 Global_Rec_Hist_Q[Global_Rec_FIR_Pos] = Shifted_Q;
1299 int newest_pos = Global_Rec_FIR_Pos;
1301 Global_Rec_FIR_Pos++;
1303 if (Global_Rec_FIR_Pos >= REC_FIR_TAPS) {
1305     Global_Rec_FIR_Pos = 0;
1307 }
```

The newest translated sample is inserted into a circular history of length $M = 255$. This avoids shifting an entire 255-element array for every new sample.

6.5 Decimation gate

```

1313 Global_Rec_Acc_Count++;
1315 if (Global_Rec_Acc_Count < Global_Rec_Decimation) {
1317     return;
1319 }
1321 Global_Rec_Acc_Count = 0;
```

Only every D th input sample produces an output sample. Therefore the output rate is approximately

$$F_{s,\text{out}} = \frac{F_s}{D}. \quad (63)$$

The function still stores every translated input sample into the FIR history; it simply avoids computing the full 255-tap convolution on input samples that will be discarded.

6.6 FIR convolution at output instants

```

1325 double Filtered_I = 0.0;
1326 double Filtered_Q = 0.0;
1328 int hist_idx = newest_pos;
1330 for (int tap = 0; tap < REC_FIR_TAPS; tap++) {
1331     Filtered_I += Global_Rec_FIR[tap] * Global_Rec_Hist_I[hist_idx];
1332     Filtered_Q += Global_Rec_FIR[tap] * Global_Rec_Hist_Q[hist_idx];
    ...
1340 }
```

For each output instant n_m , the complex FIR result is

$$y[n_m] = \sum_{r=0}^{M-1} \hat{h}[r] x'[n_m - r]. \quad (64)$$

Separating components,

$$I_y[n_m] = \sum_{r=0}^{M-1} \hat{h}[r] I'[n_m - r], \quad (65)$$

$$Q_y[n_m] = \sum_{r=0}^{M-1} \hat{h}[r] Q'[n_m - r]. \quad (66)$$

The same real-valued FIR coefficients are applied independently to I and Q. This preserves the complex analytic signal structure.

Filtering before decimation suppresses spectral content that would otherwise alias when the sample rate is reduced.

7 Complex16 Quantization and Queue Conversions

7.1 Float to signed 16-bit IQ

Both the pre-record cache, recording queue, and final recording path clamp floating-point values to $[-1, 1]$ and scale by 32767:

```

727 if (I > 1.0f) I = 1.0f;
733 if (I < -1.0f) I = -1.0f;
739 if (Q > 1.0f) Q = 1.0f;
745 if (Q < -1.0f) Q = -1.0f;
751 c->I[c->write_pos] = (int16_t)(I * 32767.0f);
752 c->Q[c->write_pos] = (int16_t)(Q * 32767.0f);

```

Mathematically,

$$q_{16} = \text{int16}(32767 \cdot \text{clip}(x, -1, 1)). \quad (67)$$

The number 32767 is the largest positive signed 16-bit integer:

$$2^{15} - 1 = 32767. \quad (68)$$

This creates a symmetric intentional output range of approximately $[-32767, +32767]$ rather than using -32768 on the negative side.

7.2 Queue/cache reconstruction back to float

Relevant code from `recorder_thread.main()`:

```

1425 float I = (float)buf_I[n] / 32768.0f;
1426 float Q = (float)buf_Q[n] / 32768.0f;
1428 recorder_write_sample(I, Q);

```

The reconstruction is

$$x_f = \frac{x_{16}}{32768}. \quad (69)$$

Because writing used 32767 while reading uses 32768, one queue/cache round trip introduces a maximum scaling factor of

$$\frac{32767}{32768} \approx 0.9999695, \quad (70)$$

which is roughly -0.000265 dB and is negligible for the intended application.

7.3 Five-second pre-record cache capacity

The pre-cache capacity is calculated as

```
626 c->capacity = (size_t)sample_rate_hz * PRE_RECORD_SECONDS;
```

so

$$N_{\text{cache}} = F_s T_{\text{cache}}, \quad (71)$$

with

$$T_{\text{cache}} = 5 \text{ s}. \quad (72)$$

At 2 MHz, this is

$$N_{\text{cache}} = 2,000,000 \times 5 = 10,000,000 \quad (73)$$

complex IQ samples.

8 End-to-End Recording Equation

For the decimating case $D > 1$, the core recording DSP can be summarized as follows.

First translate the selected frequency to DC:

$$x'[n] = x[n]e^{-j2\pi\Delta f n/F_s}. \quad (74)$$

Then low-pass filter:

$$v[n] = \sum_{r=0}^{M-1} \hat{h}[r]x'[n-r]. \quad (75)$$

Then retain one sample every D inputs:

$$y[m] = v[n_m], \quad n_{m+1} - n_m = D. \quad (76)$$

In idealized conventional notation this is commonly written as

$$y[m] = \sum_{r=0}^{M-1} \hat{h}[r]x'[mD-r], \quad (77)$$

up to the implementation's initial gate offset.

Finally quantize I and Q independently:

$$I_{16}[m] = \text{int16}(32767 \cdot \text{clip}(\Re\{y[m]\}, -1, 1)), \quad (78)$$

$$Q_{16}[m] = \text{int16}(32767 \cdot \text{clip}(\Im\{y[m]\}, -1, 1)). \quad (79)$$

9 Worked Recording Example

Consider the default receive settings

$$f_c = 101.3 \text{ MHz}, \quad (80)$$

$$F_s = 2.0 \text{ MHz}, \quad (81)$$

$$B_{\text{display}} = 1.0 \text{ MHz}. \quad (82)$$

Suppose the selector is

$$X_0 = 0.70, \quad X_1 = 0.80. \quad (83)$$

9.1 Selected center

$$x_{\text{sel}} = \frac{0.70 + 0.80}{2} = 0.75, \quad (84)$$

$$\Delta f = (0.75 - 0.5)(1,000,000) = 250,000 \text{ Hz}, \quad (85)$$

$$f_{\text{rec}} = 101.3 \text{ MHz} + 0.25 \text{ MHz} = 101.55 \text{ MHz}. \quad (86)$$

9.2 Selected bandwidth

$$B = |0.80 - 0.70|(1,000,000) \quad (87)$$

$$= 100,000 \text{ Hz} = 100 \text{ kHz}. \quad (88)$$

9.3 Target rate and decimation

$$F_{s,\text{target}} = 3B = 300,000 \text{ samples/s}, \quad (89)$$

$$D = \left\lfloor \frac{2,000,000}{300,000} \right\rfloor = 6, \quad (90)$$

$$F_{s,\text{out}} = \left\lfloor \frac{2,000,000}{6} \right\rfloor = 333,333 \text{ samples/s}. \quad (91)$$

9.4 Filter cutoff

$$B/2 = 50,000 \text{ Hz}, \quad (92)$$

$$0.45F_{s,\text{out}} \approx 149,999.85 \text{ Hz}, \quad (93)$$

$$f_c = \min(50,000, 149,999.85) = 50,000 \text{ Hz}, \quad (94)$$

$$f_n = \frac{50,000}{2,000,000} = 0.025. \quad (95)$$

9.5 Mixer phase step

$$\Delta\phi = -2\pi \frac{250,000}{2,000,000} \quad (96)$$

$$= -\frac{\pi}{4} \text{ rad/sample}. \quad (97)$$

Thus the complex local oscillator rotates by -45° per input sample, translating a signal 250 kHz above the SDR center down to DC.

10 Quarter-Wave Antenna Calculation

Although this is an RF calculation rather than a DSP operation, it is implemented in `RetroSpectrum.c` and is mathematically relevant.

Relevant code (RetroSpectrum.c lines 2617–2641):

```
2640 double wavelength_m = 299792458.0 / (double)freq_hz;
2641 double quarter_wave_m = wavelength_m / 4.0;
2643 return quarter_wave_m * 39.37007874;
```

Free-space wavelength is

$$\lambda = \frac{c}{f}, \quad (98)$$

where

$$c \approx 299,792,458 \text{ m/s}. \quad (99)$$

A quarter-wave element is

$$L_{1/4} = \frac{\lambda}{4} = \frac{c}{4f}. \quad (100)$$

The conversion to inches is

$$L_{\text{in}} = \frac{c}{4f} \times 39.37007874. \quad (101)$$

This is a free-space theoretical length; a physical antenna may require shortening due to conductor diameter, insulation, nearby materials, velocity factor, and matching geometry.

11 Transmit-Side CS16 to CF32 Conversion

The transmit thread can convert signed 16-bit IQ to floating point when the selected SoapySDR TX stream uses CF32.

```
3056 for (size_t value = 0; value < iq_count * 2U; value++) {
3057     converted_cf32[value] = (float)input[value] / 32768.0f;
3058 }
```

The conversion is

$$x_{\text{CF32}} = \frac{x_{\text{CS16}}}{32768}. \quad (102)$$

This is the same normalization convention used for incoming CS16 receive samples.

12 Engineering Notes and Interpretation

1. **Hann and Hamming serve different purposes.** The live spectrum uses a Hann window on each FFT block. The recording anti-alias filter uses a Hamming-windowed sinc FIR.
2. **The dB display is relative.** `compute_DB_from_FFT()` computes $20 \log_{10}$ of normalized FFT magnitude and then adds 100. It does not compute calibrated dBm.
3. **The FIR is only required for decimation in the current implementation.** When `Global_Rec_Decimation <= 1`, the shifted IQ is written directly without the recording FIR.
4. **The FIR operates at the input sample rate.** The normalized cutoff uses the input sample rate, `Global_Sample_Rate_Hz`, rather than the output rate. Convolution occurs on the original-rate sample history before decimated outputs are selected.
5. **Every input sample still enters the history buffer.** RetroSpectrum saves CPU by running the 255-tap convolution only at output instants, not by skipping samples before the anti-alias filter.
6. **The 0.45 factor is a safety cap.** It prevents the FIR cutoff from approaching the post-decimation Nyquist frequency. Under the normal 3x bandwidth rule, the B/2 cutoff is typically much lower and provides substantially more transition-band margin.
7. **The output-rate choice uses 3x selected bandwidth.** The target output rate is $\max(3B, 48000)$, providing sampling and transition-band headroom beyond the selected bandwidth.

8. **Initial FIR output includes filter startup transient.** The history arrays are zeroed when the recording processing state is reset, so the first FIR outputs ramp from a zero-padded initial condition until the filter history fills.
9. **The queue/cache introduces one quantization stage before recording DSP.** Live float IQ is stored as complex16 in the recording queue and reconstructed to float before `recorder_write_sample`. This reduces queue memory and keeps a consistent integer storage format, at the cost of normal 16-bit quantization noise.

13 Compact Formula Reference

Operation	Equation
Selector center	$x_{\text{sel}} = (X_0 + X_1)/2$
Frequency offset	$\Delta f = (x_{\text{sel}} - 0.5)B_{\text{display}}$
Selected RF center	$f_{\text{sel}} = f_c + \Delta f$
Selected bandwidth	$B = X_1 - X_0 B_{\text{display}}$
CS16 normalization	$x_f = x_{16}/32768$
DC estimate	$d[n] = (1 - \alpha)d[n - 1] + \alpha x[n]$
DC-blocked output	$y[n] = x[n] - d[n]$
Hann window	$w[n] = 0.5 - 0.5 \cos(2\pi n/(N - 1))$
FFT	$X[k] = \sum_{n=0}^{N-1} x[n]w[n]e^{-j2\pi kn/N}$
FFT shift	$k' = (k + N/2) \bmod N$
FFT magnitude	$A[k] = \sqrt{I_k^2 + Q_k^2}/N$
Display dB	$D[k] = 20 \log_{10}(A[k] + 10^{-12}) + 100$
Target record rate	$F_{s,\text{target}} = \max(3B, 48000)$
Decimation	$D = \max(1, \lfloor F_s/F_{s,\text{target}} \rfloor)$
Output rate	$F_{s,\text{out}} = \lfloor F_s/D \rfloor$
Record LPF cutoff	$f_c = \min(B/2, 0.45F_{s,\text{out}})$
Normalized cutoff	$f_n = f_c/F_s$
Ideal LPF center tap	$h[0] = 2f_n$
Ideal LPF other taps	$h[m] = \sin(2\pi f_n m)/(\pi m)$
Hamming window	$w_H[n] = 0.54 - 0.46 \cos(2\pi n/(M - 1))$
Normalized FIR	$\hat{h}[n] = h[n]/\sum_r h[r]$
Mixer phase step	$\Delta\phi = -2\pi\Delta f/F_s$
Complex translation	$x'[n] = x[n]e^{-j2\pi\Delta f n/F_s}$
FIR convolution	$y[n] = \sum_{r=0}^{M-1} \hat{h}[r]x'[n - r]$
Complex16 output	$q_{16} = \text{int16}(32767 \text{ clip}(x, -1, 1))$
Quarter-wave length	$L_{\text{in}} = (c/(4f)) 39.37007874$

A Full Core DSP Function Listings

The following listings reproduce the core calculation-bearing functions from the supplied source baseline for reference.

A.1 `selection_center_Hz()`

```
471 uint64_t selection_center_Hz(void) {
472     /*
473         Purpose: Computes the selected recording center frequency in Hz
474         Returns: Center frequency
475     */
476     double Center_Frac = (Global_Selector.X0 + Global_Selector.X1) * 0.5;
477     double Offset_Hz = (Center_Frac - 0.5) * (double)Global_Display_Span_Hz;
478     double Calc_Freq = (double)Global_Center_Freq_Hz + Offset_Hz;
479     if (Calc_Freq < 0.0) {
480         Calc_Freq = 0.0;
481     }
482     return (uint64_t)Calc_Freq;
483 }
```

A.2 selection_BW_Hz()

```
492 uint32_t selection_BW_Hz(void) {
493     /*
494         Purpose: Computes the selected recording bandwidth in Hz
495         Returns: Bandwidth value
496     */
497     double BW = fabs(Global_Selector.X1 - Global_Selector.X0) * (double)Global_Display_Span_Hz;
498     if (BW < 1000.0) {
499         BW = 1000.0;
500     }
501     if (BW > (double)Global_Sample_Rate_Hz) {
502         BW = (double)Global_Sample_Rate_Hz;
503     }
504     return (uint32_t)BW;
505 }
```

A.3 configure_recording_filter()

```
517 static void configure_recording_filter(void) {
518     /*
519         Purpose: Configures the FIR filter and decimation used by recording
520         Returns: No value
521     */
522     memset(Global_Rec_FIR, 0, sizeof(Global_Rec_FIR));
523     memset(Global_Rec_Hist_I, 0, sizeof(Global_Rec_Hist_I));
524     memset(Global_Rec_Hist_Q, 0, sizeof(Global_Rec_Hist_Q));
525     Global_Rec_FIR_Pos = 0;
526     Global_Rec_Acc_Count = 0;
527     // Output rate should be comfortably above selected bandwidth
528     // 2.5x gives room for FIR transition
529     double wanted_out_rate = (double)Global_Rec_BW_Hz * 3;
530     if (wanted_out_rate < 48000.0) {
531         wanted_out_rate = 48000.0;
532     }
533     Global_Rec_Decimation = (int)((double)Global_Sample_Rate_Hz / wanted_out_rate);
534     if (Global_Rec_Decimation < 1) {
535         Global_Rec_Decimation = 1;
536     }
537     Global_Rec_Out_Rate_Hz = Global_Sample_Rate_Hz / (uint32_t)Global_Rec_Decimation;
538     // After shifting selected center to 0 Hz, selected bandwidth is
539     // -BW/2 to +BW/2
540     double cutoff_hz = (double)Global_Rec_BW_Hz * 0.5;
541     // Keep cutoff below decimated Nyquist
542     double max_safe_cutoff = (double)Global_Rec_Out_Rate_Hz * 0.45;
543     if (cutoff_hz > max_safe_cutoff) {
544         cutoff_hz = max_safe_cutoff;
545     }
546     // Normalized cutoff relative to input sample rate
547     double fc = cutoff_hz / (double)Global_Sample_Rate_Hz;
548     double sum = 0.0;
```

```

571 int mid = REC_FIR_TAPS / 2;
573 for (int n = 0; n < REC_FIR_TAPS; n++) {
574     int m = n - mid;
576     double sinc;
578     if (m == 0) {
580         sinc = 2.0 * fc;
582     }
584     else {
586         sinc = sin(2.0 * M_PI * fc * (double)m) / (M_PI * (double)m);
588     }
590     // Hamming window
592     double window = 0.54 - 0.46 * cos((2.0 * M_PI * (double)n) / (double)(REC_FIR_TAPS - 1));
594     Global_Rec_FIR[n] = sinc * window;
595     sum += Global_Rec_FIR[n];
596 }
598 // Normalize gain to 1.0
600 if (fabs(sum) > 1e-12) {
602     for (int n = 0; n < REC_FIR_TAPS; n++) {
603         Global_Rec_FIR[n] /= sum;
604     }
606 }
607 }

```

A.4 ring_read_block()

```

1683 static int ring_read_block(Type_RingBuf *r, fftw_complex *in, double *window) {
1689     pthread_mutex_lock(&r->lock);
1691     if (ring_available_locked(r) < FFT_SIZE) {
1693         pthread_mutex_unlock(&r->lock);
1694         return 0;
1696     }
1698     for (int sam = 0; sam < FFT_SIZE; sam++) {
1699         size_t idx = (r->read_pos + sam) % RING_SIZE;
1700         in[sam][0] = r->I[idx] * window[sam];
1701         in[sam][1] = r->Q[idx] * window[sam];
1702     }
1703     r->read_pos = (r->read_pos + FFT_SIZE / 2) % RING_SIZE;
1704     pthread_mutex_unlock(&r->lock);
1705     return 1;
1706 }

```

A.5 compute_DB_from_FFT()

```

2374 static void compute_DB_from_FFT(fftw_complex *out, double *db) {
2380     for (int sam = 0; sam < FFT_SIZE; sam++) {
2381         int shifted_sam = (sam + FFT_SIZE / 2) % FFT_SIZE;
2383         double I = out[shifted_sam][0];
2384         double Q = out[shifted_sam][1];
2386         double magnitude = sqrt(I * I + Q * Q) / FFT_SIZE;
2387         db[sam] = 20.0 * log10(magnitude + 1e-12) + 100.0;
2388     }
2389 }

```

A.6 Core calculation portion of process_rx_samples()

```

2235 if (samples_cs16) {
2237     I = (float)samples_cs16[2U * sample] / 32768.0f;
2238     Q = (float)samples_cs16[2U * sample + 1U] / 32768.0f;
2240 }
2242 else {
2244     I = samples_cf32[2U * sample];
2245     Q = samples_cf32[2U * sample + 1U];
2247 }

```

```

2249 if (!isfinite(I)) I = 0.0f;
2255 if (!isfinite(Q)) Q = 0.0f;
2261 if (Global_DC_Enable) {
2263     const double alpha = 0.0001;
2265     Global_DC_I += alpha * ((double)I - Global_DC_I);
2266     Global_DC_Q += alpha * ((double)Q - Global_DC_Q);
2268     I -= (float)Global_DC_I;
2269     Q -= (float)Global_DC_Q;
2271 }

```

A.7 recorder_write_sample()

```

1221 static void recorder_write_sample(float I, float Q) {
1227     if (!Global_Rec_File) {
1229         return;
1231     }
1233     // Shift selected center frequency to baseband
1235     double Freq_Offset_Hz = (double)Global_Rec_Center_Hz - (double)Global_Center_Freq_Hz;
1236     double Phase_Step = -2.0 * M_PI * Freq_Offset_Hz / (double)Global_Sample_Rate_Hz;
1238     double C = cos(Global_Rec_Phase);
1239     double S = sin(Global_Rec_Phase);
1241     double Shifted_I = I * C - Q * S;
1242     double Shifted_Q = I * S + Q * C;
1244     Global_Rec_Phase += Phase_Step;
1246     if (Global_Rec_Phase > M_PI) {
1248         Global_Rec_Phase -= 2.0 * M_PI;
1250     }
1252     if (Global_Rec_Phase < -M_PI) {
1254         Global_Rec_Phase += 2.0 * M_PI;
1256     }
1258     if (Global_Rec_Decimation <= 1) {
1260         if (Shifted_I > 1.0) Shifted_I = 1.0;
1266         if (Shifted_I < -1.0) Shifted_I = -1.0;
1272         if (Shifted_Q > 1.0) Shifted_Q = 1.0;
1278         if (Shifted_Q < -1.0) Shifted_Q = -1.0;
1284         int16_t iq_pair[2];
1286         iq_pair[0] = (int16_t)(Shifted_I * 32767.0);
1287         iq_pair[1] = (int16_t)(Shifted_Q * 32767.0);
1289         fwrite(iq_pair, sizeof(int16_t), 2, Global_Rec_File);
1290         return;
1292     }
1296     Global_Rec_Hist_I[Global_Rec_FIR_Pos] = Shifted_I;
1297     Global_Rec_Hist_Q[Global_Rec_FIR_Pos] = Shifted_Q;
1299     int newest_pos = Global_Rec_FIR_Pos;
1301     Global_Rec_FIR_Pos++;
1303     if (Global_Rec_FIR_Pos >= REC_FIR_TAPS) {
1305         Global_Rec_FIR_Pos = 0;
1307     }
1313     Global_Rec_Acc_Count++;
1315     if (Global_Rec_Acc_Count < Global_Rec_Decimation) {
1317         return;
1319     }
1321     Global_Rec_Acc_Count = 0;
1325     double Filtered_I = 0.0;
1326     double Filtered_Q = 0.0;
1328     int hist_idx = newest_pos;
1330     for (int tap = 0; tap < REC_FIR_TAPS; tap++) {
1331         Filtered_I += Global_Rec_FIR[tap] * Global_Rec_Hist_I[hist_idx];
1332         Filtered_Q += Global_Rec_FIR[tap] * Global_Rec_Hist_Q[hist_idx];
1334         hist_idx--;
1336         if (hist_idx < 0) {
1338             hist_idx = REC_FIR_TAPS - 1;
1340         }
1341     }
1343     if (Filtered_I > 1.0) Filtered_I = 1.0;
1349     if (Filtered_I < -1.0) Filtered_I = -1.0;
1355     if (Filtered_Q > 1.0) Filtered_Q = 1.0;
1361     if (Filtered_Q < -1.0) Filtered_Q = -1.0;

```

```
1367     int16_t iq_pair[2];
1369     iq_pair[0] = (int16_t)(Filtered_I * 32767.0);
1370     iq_pair[1] = (int16_t)(Filtered_Q * 32767.0);
1372     fwrite(iq_pair, sizeof(int16_t), 2, Global_Rec_File);
1373 }
```

A.8 recommended_antenna_length_inches()

```
2617 double recommended_antenna_length_inches(uint64_t freq_hz) {
2623     if (freq_hz == 0) {
2625         return 0.0;
2627     }
2640     double wavelength_m = 299792458.0 / (double)freq_hz;
2641     double quarter_wave_m = wavelength_m / 4.0;
2643     return quarter_wave_m * 39.37007874;
2644 }
```